

Learning Which Item to Evict Inside an Adaptive Replacement Cache

Yash Laxman, R Ajay Kumar, Kritharth B Shetty, and Pooja J Naik

Department of Computer Science and Engineering,
Sahyadri College of Engineering & Management, Mangaluru, India

Abstract. Every cache has to decide which item to remove when it runs out of space. Classic rules such as LRU and LFU each work well on some workloads and poorly on others. ARC improves on them by balancing recency against frequency, but once it has picked a list to shrink it always removes the oldest item in that list. This chapter presents TIDE, a design that keeps ARC exactly as it is and hands the final eviction choice to a feedforward neural network trained continually on live traffic. The network regresses, for each of at most six candidates, a scaled logarithm of the time until the item is requested again, which is the quantity Belady’s optimal rule ranks by. It is trained online by backpropagation and Adam from delayed and right-censored labels that the request stream produces on its own, replayed through an experience buffer, with no offline dataset and no reward signal. The network may also decide not to cache a new item at all, and a shadow copy of ARC takes control back when the network is wrong. Across 43 trace and cache-size settings, TIDE has the best mean normalised hit rate among six policies (0.379 against 0.334 for S3-FIFO and 0.306 for ARC), gains up to 34.5 points over ARC on workloads with learnable structure, and never trails ARC by more than 0.54 points.

Keywords: Deep learning · Neural networks · Online learning · Cache replacement · Adaptive replacement cache · Eviction policy

1 Introduction

A cache keeps a small number of items close at hand so that repeated requests can be served quickly. When the cache is full and a new item arrives, something has to go. The rule that makes this choice is the replacement policy, and it decides the hit rate: the share of requests the cache can answer without going to slower storage. Even a few points of hit rate can matter a great deal, because every miss costs a disk read, a database query or a network trip.

The best possible rule has been known for a long time. Belady showed that a cache should remove the item that will be needed furthest in the future [2]. Real systems cannot see the future, so they guess. LRU guesses that recently used items will be used again soon. LFU guesses that popular items will stay popular. Each guess holds on some workloads and fails on others.

Learned policies try to replace these fixed guesses with a model trained on real traffic [12, 9]. In practice they tend to throw away the heuristic they replace, they add cost to every decision, and they can do much worse than the old rule when the model is wrong. We take a narrower path. ARC [10] already does one thing well: it decides whether the recency side or the frequency side of the cache should shrink. It does a second thing poorly: inside the chosen side it always removes the oldest item. TIDE keeps the first decision and hands the second to a neural network.

The learning problem is the centre of this chapter. The network sees a handful of candidates, each described by twelve numbers, and must predict how far away each candidate’s next request is. It never sees a labelled dataset: the correct answer for each prediction arrives later, when the item is requested again, or never arrives, in which case it must be treated as a censored observation. We show that a deliberately compact network, one hidden layer of 64 rectified linear units, trained continually by backpropagation on these delayed labels and stabilised by an experience replay buffer, is enough to learn periodic and phase-dependent reuse patterns that no fixed heuristic can see.

This chapter makes four contributions. It frames eviction inside ARC as a regression problem over a small, fixed set of candidates. It presents the network, its input representation, its target and its online training procedure in full. It gives simple arguments for how the design should behave, including its cost and how far it can fall behind ARC. Finally, it tests those arguments on fifteen workloads against five classic policies.

2 Related Work

2.1 Belady’s Rule

Belady’s rule removes the item whose next request is furthest away, and no other demand paging policy can have fewer misses on the same sequence [2]. It is used as a yardstick. A common way to report results is a normalised hit rate, where 0 means no better than LRU and 1 means as good as Belady’s rule. When requests are drawn at random from a fixed set of popularities, the best rule that does not know the future is simply to keep the most popular items [1].

2.2 Classic Policies

Table 1 lists the classic policies that matter most here. Each one relies on a single idea about the workload, and each has a case where that idea breaks.

2.3 How ARC Works

ARC splits the cache into two lists. T_1 holds items seen once recently and T_2 holds items seen at least twice. It also remembers the keys of recently removed items in two ghost lists, B_1 and B_2 , without keeping their data. If a request hits

Table 1. Classic replacement policies.

Policy	Main signal	Where it struggles
LRU	Time since last use	Scans and loops longer than the cache
LFU	Number of uses	Holds on to items that are no longer popular
2Q [6]	Trial queue, then LRU	Queue sizes are fixed in advance
LIRS [5]	Gap between uses	Extra bookkeeping and tuning
ARC [10]	Recency and frequency lists	Always removes the oldest item of a list
S3-FIFO [14]	Small and main FIFO queues	No view of when an item returns
SIEVE [15]	FIFO with a visited bit	Only a yes or no view of reuse

a ghost list, ARC takes it as a sign that it shrank the wrong list, and it moves a target size p in the other direction. When space is needed, ARC removes an item from T_1 if T_1 is larger than p and from T_2 otherwise.

This choice of list is ARC’s strength. Its weakness is what happens next: it always removes the least recently used item of that list. On a loop that is slightly longer than the cache, the oldest item is exactly the one that will be needed next, so ARC gets no hits at all.

2.4 Learned Policies

Several groups have trained models to make eviction decisions (Table 2). Hawkeye learns from Belady’s rule replayed on past requests [4]. LRB trains gradient-boosted trees online for content delivery caches [12]. Parrot imitates Belady’s rule with a deep network trained offline, and reports that imitation outperforms a reinforcement learning formulation of the same task [9]. CACHEUS mixes the advice of two heuristics [11]. HALP is the closest to our idea: a heuristic suggests a few candidates and a small neural network with one hidden layer picks one [13]. None of them uses ARC’s choice of list as a starting point.

Table 2. Learned replacement policies.

Policy	Learns from	Role of a heuristic
Hawkeye	Belady’s rule on past requests	None
LRB	Relaxed Belady rule, online	None
Parrot	Belady’s rule, offline	None
CACHEUS	Regret on removed items	Mixes two heuristics
HALP	Preference feedback, online	Suggests candidates
TIDE	Time until next request, online	ARC picks the list and is the fallback

3 Problem Formulation

We want eviction choices that are closer to Belady’s rule without giving up what ARC already does well. We also want the extra cost of each decision to stay small and fixed, and we want the design to fall back to ARC when the model is wrong.

The key quantity is the reuse distance d of an item: how many requests from now it will be asked for again. Belady’s rule removes the item with the largest d . We do not need the exact value, only the order of the candidates, so the network predicts

$$g(d) = \log \left(1 + \frac{d}{c} \right), \quad (1)$$

where c is the cache size. Dividing by c puts caches of different sizes on the same scale, and the logarithm stops a few very large distances from dominating the squared-error loss. Because g is monotone, ranking candidates by $g(d)$ is the same as ranking them by d , so a perfect regressor reproduces Belady’s choice among the candidates it is shown.

We treat this as regression rather than classification on purpose. A classifier that predicts whether an item will be hit if kept would need a calibrated probability for each candidate and a further rule to rank them. A regressor on $g(d)$ gives a ranking directly, and its target is exactly the quantity the optimal offline rule uses.

4 Proposed Method

4.1 Overview

TIDE keeps all of ARC: its four lists, its target size and its choice of which list to shrink. The only change is the last step. Instead of removing the oldest item of the chosen list, TIDE scores a few candidates with the network and removes the one it expects to be needed last. Figure 1 shows the parts of the design: a decision path that runs on every miss, and a learning path that turns what happens next into training data.

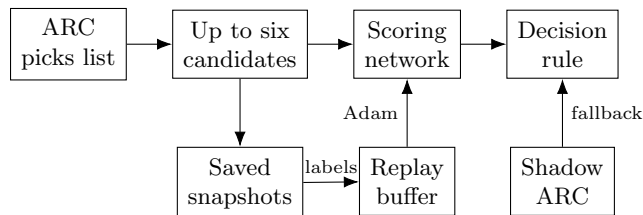


Fig. 1. The parts of TIDE. ARC picks the list, the network ranks a few candidates, snapshots of every scored candidate become training examples once their outcome is known, and a shadow copy of ARC decides when to hand control back.

4.2 Candidates

For the list ARC has chosen, TIDE looks at up to six items:

- the four oldest items, where the oldest one is ARC’s own choice;
- the newest item, which is often the right one to drop on long loops, because it will not come back until the loop comes round again;
- on a miss for a new item when the cache is full, the item being requested, which lets TIDE serve it without storing it. We call this a bypass. It protects the cache from one-time scans.

The number of candidates never depends on the cache size. Limiting the network to the tail of the list follows LRB and HALP, which found that a few cold candidates capture most of the benefit [12, 13].

4.3 Input Representation

TIDE keeps a short history for each item it has seen recently, including items that were removed, so it can recognise them when they come back. This history is capped at $8c$ entries, and items that are in the cache always keep theirs. From the history it builds the twelve numbers in Table 3, which form the input vector $x \in \mathbb{R}^{12}$. Every time value is measured in requests, divided by c and passed through $\log(1 + \cdot)$, the same transform as the target, so that the same input means the same thing whether the cache holds 16 items or 20,000. A gap that has not yet been observed, for an item seen fewer than four times, is filled with the sentinel $\log(1 + 2W/c)$ used for censored labels in Sect. 4.5. None of the features look at the content of an item, so the design works with any kind of key.

Table 3. The twelve input features computed for each candidate. Time values are scaled as $\log(1 + t/c)$.

No.	Feature	What it captures
1	Time since last request	Recency
2 to 4	Last three gaps between requests	Regular patterns and loops
5	Smoothed average gap	Typical return time
6	Request count, capped at 3 and scaled to $[0, 1]$	Whether the item is new or repeat
7 to 9	Three decaying counters, half-lives $c, 4c, 16c$	Popularity over short, medium and long spans
10	$\log(1 + \text{total request count})$	Long term popularity
11	Is this the requested item?	Marks the bypass option
12	Is it in the frequency list?	ARC’s own view of the item

As an example, take $c = 100$ and an item last requested 40 requests ago, whose three previous gaps were 95, 105 and 98 requests. Its first four features

are $\log 1.40 \approx 0.34$, $\log 1.95 \approx 0.67$, $\log 2.05 \approx 0.72$ and $\log 1.98 \approx 0.68$. Three nearly equal gaps tell the network that this item returns on a clock of about 100 requests, which a feature based on recency alone cannot express.

There is no feature for trend. An item that is gaining popularity already shows shorter gaps and a short term counter that is higher than its long term one, and the network can learn to read that combination.

4.4 Neural Network Architecture

The scoring network is a multilayer perceptron with one hidden layer of 64 rectified linear units and a linear output (Fig. 2). For a candidate with input $x \in \mathbb{R}^{12}$ it computes

$$h = \text{ReLU}(W_1 x + b_1), \quad W_1 \in \mathbb{R}^{64 \times 12}, \quad b_1 \in \mathbb{R}^{64}, \quad (2)$$

$$\hat{y} = w_2^\top h + b_2, \quad w_2 \in \mathbb{R}^{64}, \quad b_2 \in \mathbb{R}, \quad (3)$$

where $\hat{y}$ is the predicted $g(d)$. The network has $12 \cdot 64 + 64 + 64 + 1 = 897$ trainable parameters. All candidates of a decision are scored together as one batch of at most six rows, and the victim is chosen from the six outputs by the decision rule of Sect. 4.6.

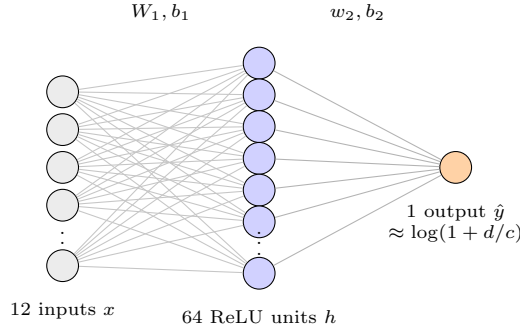


Fig. 2. The scoring network: twelve features, one hidden layer of 64 rectified linear units, and a linear output that predicts the scaled reuse distance.

Initialisation and optimisation. W_1 is drawn from a zero-mean normal distribution with variance $2/12$, the He initialisation for rectified units [3], and w_2 with variance $1/64$. Biases start at zero, except that just before the first gradient step the output bias b_2 is set to the mean of the labels collected so far. The network therefore starts by predicting the average reuse distance and only has to learn deviations from it, which removes a large initial transient from the loss. Training minimises the mean squared error between $\hat{y}$ and $g(d)$ with Adam [7]

at a learning rate of 10^{-3} and $\beta_1 = 0.9$, $\beta_2 = 0.999$. Adam’s per-parameter step sizes matter here because the data stream is not stationary: when the workload changes phase, the distribution of both inputs and targets shifts, and an adaptive optimiser recovers faster than a fixed step.

Why a small network. The network is small by design, for three reasons. First, it runs on every miss, inside a latency budget of microseconds, and it only ever ranks at most six candidates, so extra capacity would add cost on every decision. Second, the features of Table 3 already summarise the part of an item’s history that matters for its next reuse: its recent gaps, their smoothed average and its popularity at three time scales. A sequence model such as a recurrent or temporal convolutional network would have to learn this summary from raw request histories at a much higher cost per decision, and we have not found a reason to expect it to rank six candidates better. Third, the quality of the labels, not the expressiveness of the model, is the main limit on what can be learned online. The same conclusion was reached for HALP, which uses a network of the same shape in production [13]. What makes TIDE a learning system is therefore not the depth of its network but the way it is trained, described next.

4.5 Online Training from Delayed Labels

The network is trained continually while the cache runs, with no offline dataset. At decision time the true reuse distance of a candidate is unknown by definition, so TIDE uses delayed labelling. Each time a candidate is scored, a snapshot of its feature vector and the current time t_0 is saved. If the item is requested again at time t_1 , the snapshot is labelled $g(t_1 - t_0)$. If it has not come back after a window of $W = 8c$ requests, the observation is right-censored: TIDE knows only that $d > W$. Dropping such examples would throw away the clearest evidence that an item was not worth keeping, so they are given the fixed label $\log(1 + 2W/c) = \log 17 \approx 2.83$, as if the item had returned after twice the window. Snapshots are taken for every scored candidate whatever the decision was, because the label depends only on when the item is requested next and not on what the cache did.

Labelled examples go into a circular replay buffer of 32,768 rows. After every 32 new labels, the network takes one Adam step on a minibatch of 256 rows drawn uniformly at random from the buffer. This is the experience replay mechanism introduced for reinforcement learning agents [8], used here for the same reason: sampling at random breaks the strong correlation between consecutive requests and keeps gradient estimates stable. The finite buffer also acts as a sliding memory. Old examples are overwritten, so patterns from an earlier phase of the workload fade out of the training set as new ones arrive. Training every 32 labels rather than every request limits its share of the running time.

The network is not used to make decisions until at least W requests have been seen and at least 2,048 labels have been collected. Until then every decision is ARC’s, and the network only takes over once it has seen enough outcomes to have learned a meaningful mapping from features to reuse distance.

4.6 Decision Rule

TIDE removes the candidate with the highest score, with one condition: any candidate other than ARC’s own choice has to win by a margin. For the other old items the margin is 0.1 on the scale of g , which means the network must expect the item to return about 10% later, in units of $1 + d/c$, than ARC’s choice. For the newest item and the bypass option the margin is 0.2, or about 22% later, because these moves are riskier. When the network is unsure, ARC’s choice stands. The margin turns “is this candidate better” into “is this candidate clearly better”, which keeps a slightly noisy network from second-guessing good ARC decisions.

4.7 Fallback

A second copy of ARC, the shadow, sees the same requests and keeps only keys. TIDE counts its misses and the shadow’s misses over windows of $\max(2000, 4c)$ requests. If TIDE misses more than $\max(10, 1\%)$ of the window more than the shadow, it stops using the network and lets ARC decide for a while. The pause starts at two windows and doubles after each repeated failure, up to 32 windows. A window in which TIDE is at least as good as the shadow reduces the failure count again. Training carries on during the pause, because labels do not depend on who made the decision. Table 4 lists all settings. They were fixed before testing and are the same for every workload.

Table 4. Design settings, fixed in advance for all workloads.

Setting	Value	Reason
Candidates per decision	Up to 6	Keeps cost fixed
Network	$12 \rightarrow 64 \rightarrow 1$, ReLU	897 parameters
Optimiser	Adam, rate 10^{-3}	Adapts to non-stationary data
Loss	Squared error on $g(d)$	Ranking comes for free
Label window W	$8c$	Covers several cache turnovers
Replay buffer	32,768 rows	Old patterns fade out
Training step	Every 32 labels	Limits training time
Batch size	256	Stable updates
Margins	0.1 and 0.2	Network must be clearly better
Start after	W requests, 2,048 labels	No untrained decisions
Fallback window	$\max(2000, 4c)$	Enough requests to compare
Fallback tolerance	$\max(10, 1\%)$ of window	Small losses are ignored

5 Theoretical Analysis

This section gives short arguments for how the design should behave. We assume that the cache does not change which items users request.

5.1 Labels Come for Free

Every scored candidate gets exactly one label within W requests. The label depends only on when the item is requested again, not on what the cache did. This is the main difference from reinforcement learning. The cache never has to try bad moves to learn, and it never has to work out which of its past actions led to a later miss. From the network’s point of view, training is ordinary supervised regression, only with labels that arrive late and are sometimes censored.

5.2 Cost Stays Fixed

A hit does not use the network at all. A miss runs one forward pass over at most six candidates, which costs $6 \times (12 \cdot 64 + 64) \approx 5,000$ multiply and add operations. A training step, once every 32 labels, is a forward and backward pass over 256 rows, a few hundred thousand operations spread over at least 32 requests. The replay buffer takes about 1.7 MB, and the network with its Adam state takes about 11 kB, no matter how big the cache is. Only the per-item history grows with the cache, in the same way that ARC’s ghost lists do. The cost of a decision therefore does not grow as the cache gets bigger.

5.3 Loops Longer Than the Cache

Take a cache of c items and a workload that requests keys 1 to L in order, over and over, with $c < L$. LRU always removes the item that is needed next, so it never hits. FIFO does the same, and so does ARC. Belady’s rule removes the most recently used item, which is the one needed furthest ahead, and reaches a hit rate of $(c - 1)/(L - 1)$. A policy that keeps c keys fixed and never stores the others does better, with a hit rate of c/L , and no policy can beat it because at least $L - c$ of the keys must miss in every round. For $c = 16$ and $L = 17$ these are 93.75% and 94.1%; for $c = 16$ and $L = 31$ they are 50.0% and 51.6%.

TIDE has both of the moves needed here. It can remove the newest item, which is what Belady’s rule does, and it can skip storing the requested item, which is what the bypass policy does. ARC can do neither.

5.4 When There Is Nothing to Learn

If every request is picked at random from N equally likely items, the chance of a hit is c/N for any policy that keeps the cache full, because the next request has nothing to do with what is in the cache. TIDE cannot beat ARC here, and it should not lose either. When some items are more popular than others but requests are still random, the best possible policy keeps the most popular items [1]. TIDE can only gain the difference between ARC and that best policy, which is usually small.

5.5 How Far It Can Fall Behind ARC

The fallback limits how much worse than ARC TIDE can be over a long run. Suppose the network keeps failing. After a few failures, each window in which the network is used is followed by 32 windows in which ARC decides, so at most about one window in 33 is lost. That caps the long run loss at about 3% of ARC’s hits. A workload that switches back and forth between fooling the network and suiting it perfectly could push the loss up to about a quarter of ARC’s hits, because a single good window resets the pause. Requiring several good windows in a row before trusting the network again would close this gap. Both figures assume that once ARC is back in control, the two caches quickly end up holding similar items.

6 Experimental Evaluation

6.1 Setup

Table 5 summarises the setup. The twelve smaller traces cover fixed-popularity workloads (uniform, Gaussian, five Zipf skews, and a Zipf trace of 100,000 requests), a synthetic web trace, bursty and periodic traffic, and an adversarial trace that loops over 31 keys. The three larger traces were built to contain structure that changes over time: in *lifecycle*, items are born at random times and their popularity rises and decays; in *cycle*, 40 groups of 100 keys recur in a fixed rotating order, one group every 4,000 requests, over 30% Zipf background noise; in *phases*, each round runs one Zipf mix, a one-time scan, a loop sized to thrash a 1% cache and a second, permuted Zipf mix. Where a small trace rounds two sizes to the same c , that setting is counted once, which gives 43 settings in total.

Results are reported as the normalised hit rate $(r - r_{\text{LRU}})/(r_{\text{Belady}} - r_{\text{LRU}})$, averaged over the 42 settings in which Belady’s rule beats LRU by more than half a point. We also report the *oracle*: TIDE’s own candidates and decision rule, but scored with the true reuse distance instead of the network. The gap between TIDE and the oracle measures what the network fails to learn; the gap between the oracle and Belady’s rule measures what the choice of candidates gives up.

6.2 Results

Table 6 gives the overall ranking. TIDE has the best mean normalised hit rate and the best hit rate in 24 of the 43 settings, and it is at least as good as ARC in 34. Figure 3 shows where its gains come from.

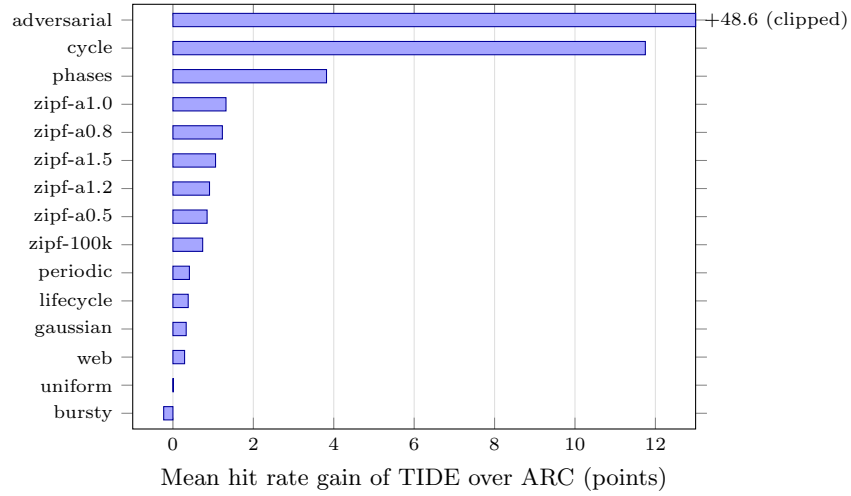
Table 7 shows selected settings. On the adversarial loop, every recency-based policy gets no hits at all, while TIDE reaches 48.55%, close to Belady’s 49.99%. The oracle reaches 51.60%, which is exactly the $c/L = 16/31$ bound of Sect. 5.3, because it can bypass. On *cycle* at 10%, the cache can finally hold about one group, and the network learns the 4,000-request period from the gap features: TIDE reaches 59.52% against 25.04% for ARC, a gain of 34.5 points, and matches

Table 5. Experimental setup.

Item	Setting
Workloads	12 traces of 100,000 requests and 3 trend traces of 1,000,000 requests
Cache sizes	0.1%, 1% and 10% of the distinct keys, at least 16
Baselines	LRU, LFU, ARC, S3-FIFO and SIEVE
Reference points	Belady’s rule, and the TIDE oracle
Repeats	TIDE averaged over 3 network seeds
Measures	Normalised hit rate, settings won, change against ARC, decision time, training time
Implementation	Python and NumPy, one CPU core per run

Table 6. Overall ranking across all 43 settings. Wins count settings in which a policy has the highest hit rate of the six.

Policy	Normalised	Wins	Mean hit rate (%)
TIDE	0.379	24	46.09
S3-FIFO	0.334	11	44.00
ARC	0.306	3	43.36
SIEVE	0.277	2	42.92
LRU	0.000	1	38.85
LFU	-0.257	2	35.24

**Fig. 3.** Gain of TIDE over ARC per trace, averaged over the three cache sizes. Workloads with structure over time (adversarial, cycle, phases) gain the most; fixed-popularity workloads gain about a point or less.

the oracle’s 59.54%. On *phases* at 1%, where the embedded loop thrashes ARC, TIDE gains 10.6 points. On fixed-popularity traces the gains are small, as Sect. 5 predicted, and on uniform traffic TIDE and ARC tie.

Table 7. Hit rate (%) in selected settings. Size is the cache size as a share of distinct keys.

Trace	Size	LRU	ARC	S3-FIFO	TIDE	Oracle	Belady
adversarial	0.1%	0.00	0.00	0.00	48.55	51.60	49.99
cycle	1%	16.13	22.87	22.76	23.50	23.61	25.27
cycle	10%	21.65	25.04	40.35	59.52	59.54	67.99
phases	1%	35.76	39.65	39.50	50.23	56.89	63.37
phases	10%	73.09	72.93	73.27	72.39	74.15	77.96
zipf-100k	1%	59.75	67.63	67.73	67.97	68.59	71.70
uniform	1%	1.01	1.04	1.05	1.04	8.53	13.14
bursty	10%	85.70	85.72	85.71	85.34	87.25	89.44

The worst setting is *phases* at 10%, where TIDE trails ARC by 0.54 points. There, ARC is already about five points from Belady’s rule, little structure is left to learn, and the network’s prediction noise costs slightly more than it gains. The fallback keeps this loss small. The mild loss on *bursty* has the same cause.

Table 8 reports speed. The median decision takes about 14 to 15 μ s and does not grow with the cache size, from $c = 50$ to $c = 2,018$. Training takes 19 to 27% of the running time. TIDE runs 20 to 70 times slower than ARC in this prototype. Almost all of this gap is interpreter and library call overhead around the small amount of arithmetic of Sect. 5, not the arithmetic itself.

Table 8. Throughput and decision latency, measured serially in Python and NumPy. Latency covers features and inference; training is reported separately as a share of running time.

Trace	c	ARC req/s	TIDE req/s	p50 (μ s)	p99 (μ s)	Training %
zipf-100k	185	3,995k	110k	14.6	22.9	21
web	50	1,864k	45k	14.8	20.2	19
lifecycle	708	3,709k	123k	15.2	30.0	20
cycle	273	2,208k	76k	13.9	21.9	27
phases	2,018	2,516k	84k	14.3	21.8	19

6.3 Discussion

The results match the analysis. The network earns its keep in proportion to how much learnable structure a workload has: large gains where recency is misleading

(loops and phase changes), about a point on skewed random traffic, and nothing on uniform traffic, where there is nothing to predict. The oracle column separates the two sources of shortfall. On *cycle* and on the adversarial loop, TIDE is within 0.1 and 3.1 points of the oracle, so the network has learned nearly all that its candidates allow. On *phases* at 1% it is 6.7 points short of the oracle, which shows that relearning after each change of phase is the harder part of the problem. On uniform and Gaussian traffic, the oracle is far above TIDE only because it knows the future; no features of the past could close that gap.

7 Limitations

The loss bound assumes that TIDE and the shadow quickly hold similar items once ARC is back in control, which is only roughly true. The design assumes that every item has the same size. Items that return after more than W requests all get the same censored label, so the network cannot tell them apart. The candidates the network sees depend on its own earlier choices, so what it trains on may drift from what it is asked to judge. All settings are fixed in advance and are not tuned per workload. Training takes up to 27% of the running time, more than we would like, and the Python prototype is 20 to 70 times slower than ARC; a compiled implementation of the feature extraction and the network is needed before the design can be used in a real cache. Finally, most of the workloads are synthetic, and TIDE was averaged over only three seeds.

8 Conclusion

TIDE keeps ARC’s choice of which list to shrink and hands the choice of which item to remove to a compact neural network, with the option of not caching a new item at all. The network regresses a scaled reuse distance, the quantity Belady’s rule ranks by, and it is trained continually by backpropagation from labels that the request stream produces on its own, handled as delayed and censored observations and replayed through an experience buffer. It needs no offline data and no trial and error. The cost of each decision stays fixed, and a shadow copy of ARC limits how much the network can hurt. Across 43 settings, TIDE has the best mean normalised hit rate of six policies, gains up to 34.5 points over ARC where workloads have structure over time, and never trails ARC by more than 0.54 points. A compiled implementation, variable item sizes and real production traces are the natural next steps.

References

1. Aho, A.V., Denning, P.J., Ullman, J.D.: Principles of optimal page replacement. *Journal of the ACM* **18**(1), 80–93 (1971)
2. Belady, L.A.: A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal* **5**(2), 78–101 (1966)

3. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 1026–1034 (2015)
4. Jain, A., Lin, C.: Back to the future: Leveraging Belady’s algorithm for improved cache replacement. In: Proceedings of the 43rd International Symposium on Computer Architecture (ISCA), pp. 78–89 (2016)
5. Jiang, S., Zhang, X.: LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. In: Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, pp. 31–42 (2002)
6. Johnson, T., Shasha, D.: 2Q: A low overhead high performance buffer management replacement algorithm. In: Proceedings of the 20th International Conference on Very Large Data Bases, pp. 439–450 (1994)
7. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: Proceedings of the 3rd International Conference on Learning Representations (ICLR) (2015)
8. Lin, L.J.: Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning* **8**(3–4), 293–321 (1992)
9. Liu, E.Z., Hashemi, M., Swersky, K., Ranganathan, P., Ahn, J.: An imitation learning approach for cache replacement. In: Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 6237–6247 (2020)
10. Megiddo, N., Modha, D.S.: ARC: A self-tuning, low overhead replacement cache. In: Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST), pp. 115–130 (2003)
11. Rodriguez, L.V., Yusuf, F., Lyons, S., Paz, E., Rangaswami, R., Liu, J., Zhao, M., Narasimhan, G.: Learning cache replacement with CACHEUS. In: Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST), pp. 341–354 (2021)
12. Song, Z., Berger, D.S., Li, K., Lloyd, W.: Learning relaxed Belady for content distribution network caching. In: Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 529–544 (2020)
13. Song, Z., Chen, K., Sarda, N., Altmöken, D., Brevdo, E., Coleman, J., Ju, X., Jurczyk, P., Schooler, R., Gummadi, R.: HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network. In: Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 1149–1163 (2023)
14. Yang, J., Zhang, Y., Qiu, Z., Yue, Y., Vinayak, R.: FIFO queues are all you need for cache eviction. In: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), pp. 130–149 (2023)
15. Zhang, Y., Yang, J., Yue, Y., Vigfusson, Y., Rashmi, K.V.: SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches. In: Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI) (2024)